Pedidos realizados en el último año: pedidos.fechaRealizacion >= DATE_SUB(CURDATE(), INTERVAL 1 YEAR)

EDAD DE UN CLIENTE: SELECT TIMESTAMPDIFF(YEAR, clientes.fechaNacimiento, CURDATE()) INTO clienteEdad
FROM pedidos
JOIN clientes ON pedidos.clienteId = clientes.id
WHERE pedidos.id = NEW.pedidoId;

fecha y la hora exacta de este mismo segundo = NOW(). EJEMPLO DE USO: WHERE NOW() BETWEEN promociones.fechaInicio AND promociones.fechaFin;

Ordenar de mayor a menor algo: ORDER BY X DESC;

Ordenar de menor a mayor algo: ORDER BY X ASC;

Quitar NULL y cambiarlo por 0 para que no deproblemas: SELECT COALESCE(SUM(unidades * precio), 0)

```
-- ——— CABECERA DE SEGURIDAD (Al principio del archivo) ———
SET FOREIGN_KEY_CHECKS = 0; -- 1. Apaga el "radar" de las claves ajenas

-- Opcional: Si quieres reiniciar la base de datos por completo en cada ejecución
DROP DATABASE IF EXISTS mi_tienda_examen;
CREATE DATABASE mi_tienda_examen;
USE mi_tienda_examen;
--
┌──────────────────────────────────────────────────────────────────────────────┐
-- │  AQUÍ PEGAS TODOS TUS CREATE TABLE ...                                   │
--
└──────────────────────────────────────────────────────────────────────────────┘

-- ——— PIE DE PÁGINA (Al final del archivo, abajo del todo) ———
SET FOREIGN_KEY_CHECKS = 1; -- 2. Vuelve a encender el radar
```

¿Cómo sé cuándo poner un UNIQUE compuesto en el examen?

Anteriormente aprendimos a detectar el UNIQUE de una sola columna (para relaciones 1 : 1, como un producto con una sola garantía).

El UNIQUE compuesto (de dos o más columnas) se detecta de otra forma. Tienes que activar las alarmas cuando veas que el enunciado describe una relación de Muchos a Muchos (*N : M*) pero le pone un límite de repetición.

Busca siempre estas "frases chivatas" en el papel:

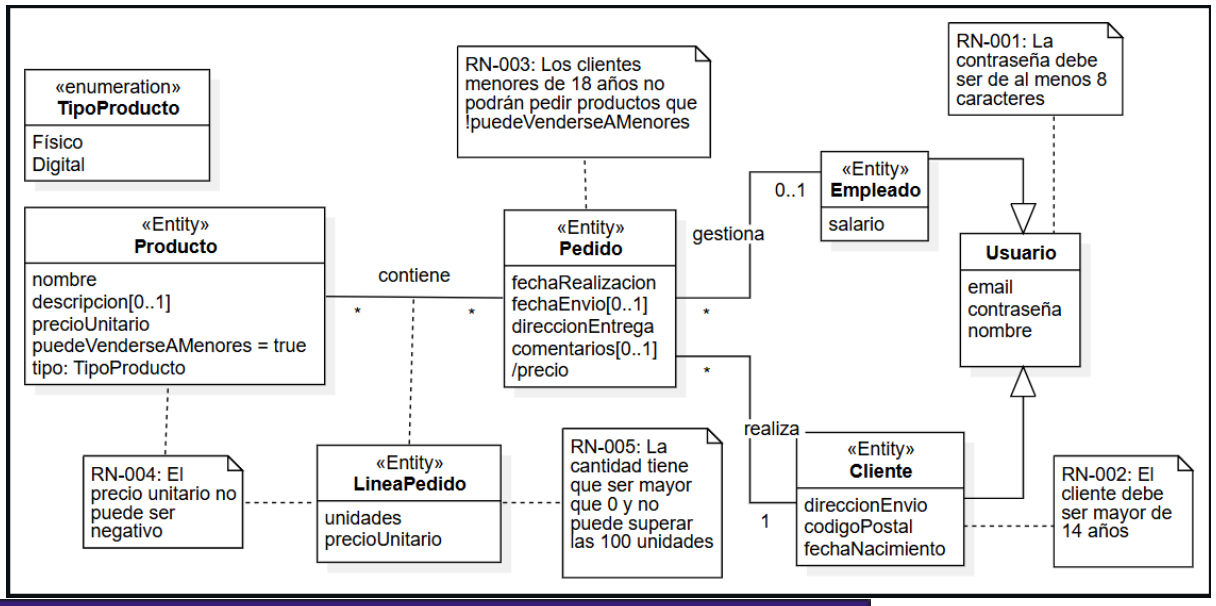
- "No se puede [hacer la acción] más de una vez sobre el mismo [objeto]": Como en tu ejercicio (*"un cliente no puede valorar más de una vez un mismo producto"*).
- "La combinación de X e Y debe ser única": Te están cantando el código directamente.
- "Un alumno solo puede matricularse una vez en una misma asignatura": Generaría un UNIQUE(alumnoId, asignaturaId).
- "Un jugador no puede pertenecer más de una vez al mismo equipo en la misma temporada": Generaría un UNIQUE(jugadorId, equipoId, temporadaId).

Si el enunciado dice...	**Objetivo real**	**Combinación ganadora**
"Impedir que un cliente menor de edad compre..."	Validar un registro nuevo antes de que entre.	BEFORE INSERT
"Calcular el descuento y aplicarlo al precio de la línea..."	Modificar datos de la propia fila que se está insertando.	BEFORE INSERT
"Impedir que un pedido se envíe si no tiene productos..."	Validar una modificación antes de que se guarde el cambio.	BEFORE UPDATE
"Al vender un producto, restar las unidades del almacén..."	Modificar **otra** tabla tras confirmar que la venta es real.	AFTER INSERT
"Si el stock baja de 10, generar una compra al proveedor..."	Crear registros en **otra** tabla tras modificarse la principal.	AFTER UPDATE

NOW() -> FECHA + HORA ACTUAL (DATETIME)
CURRENT_DATE -> FECHA SOLO ACTUAL (DATE)

Actualizar algo relacionado con otra entidad:

```
UPDATE productos JOIN lineaspedido ON productos.id = lineaspedido.productoId
SET productos.stock = productos.stock + lineaspedido.unidades
WHERE lineaspedido.pedidoId = p_pedidoId;
```




```
CREATE OR REPLACE VIEW v_resumen_pedidos_pendientes AS
SELECT
    p.id AS pedidoId,
    u.nombre AS nombreCliente,
    p.fechaEnvio,
    e.estado AS estadoEnvio
FROM usuarios u
JOIN clientes c ON u.id = c.usuarioId
JOIN pedidos p ON c.id = p.clienteId
LEFT JOIN envios e ON p.envioId = e.id
WHERE e.id IS NULL OR e.estado NOT LIKE 'Entregado';
```

A partir de ahora, para la base de datos v_resumen_pedidos_pendientes actúa exactamente como si fuera una tabla normal. Puedes hacerle filtros directos:

```
SELECT * FROM v_resumen_pedidos_pendientes
WHERE nombreCliente LIKE 'Laura%';
```

```
DELIMITER //

CREATE OR REPLACE FUNCTION f_calcular_total_pedido(p_pedido_id INT)
RETURNS DECIMAL(10,2)
READS SQL DATA -- Obligatorio en MySQL para indicarle que solo va a leer datos
BEGIN
    DECLARE v_total DECIMAL(10,2) DEFAULT 0.00;

    -- Calculamos el sumatorio total para ese pedido concreto
    SELECT SUM(lp.cantidad * pr.precio) INTO v_total
    FROM LineaPedido lp
    JOIN Producto pr ON lp.productoId = pr.id
    WHERE lp.pedidoId = p_pedido_id;

    -- Devolvemos el resultado final (Si el pedido no existe o da NULL, devolvemos 0)
    RETURN IFNULL(v_total, 0.00);
END //

DELIMITER ;
```

Aquí tienes la transcripción del texto de la imagen image_e9d1aa.png:

1. SELECT -> ¿Qué columnas quieres mostrar en la pantalla? (Campos, funciones, COUNT, SUM ...).
2. FROM -> ¿Cuál es la tabla principal de la que arrancamos?
3. JOIN -> ¿Con qué otras tablas nos tenemos que conectar? (Y sus respectivos ON).
4. WHERE -> ¿Qué filas individuales queremos filtrar **antes** de hacer cuentas? (Filtros fila por fila).
5. GROUP BY -> ¿Cómo queremos agrupar los datos? (Por id de cliente, por tipo de producto...).
6. HAVING -> ¿Qué grupos queremos filtrar **después** de haber calculado los totales? (Filtros de agregación como SUM() > 500).
7. ORDER BY -> ¿Cómo queremos ordenar el informe final? (ASC o DESC).
8. LIMIT -> ¿Cuántas filas como máximo queremos que salgan en pantalla? (Por ejemplo, el "Top 5").

```
1. Actualizar el stock de los productos según las líneas del pedido
UPDATE Productos p
JOIN LineasPedido lp ON p.id = lp.productoId
SET p.stock = p.stock + lp.unidades
WHERE lp.pedidoId = p_pedidoId;
```

SQL NOTES
VER TEMAS 10 Y 11 DE TEORÍA

Foreign Keys (4 opciones):

1. RESTRICT (por defecto): bloquea la operación
2. CASCADE: Propaga la acción. Si borras/modificas el padre, se borra/modifica automáticamente el hijo.
3. SET NULL: Permite borrar/modificar el padre, pero a los hijos les pone un valor NULL en la columna de la clave foránea.
4. SET DEFAULT: Pone el valor por defecto que tenga asignado esa columna.

NOTA: Las restricciones son sobre la tabla actual al eliminar/actualizar la de la FK

Tablas

- VARCHAR(255): String
- TEXT.
- INT: enteros
- DECIMAL(10, 2): máximo 10 dígitos, y 2 caracteres reservados para la parte decimal, siendo el máximo número el 99999999,99 (si metes un tercer decimal, MariaDB lo redondeará automáticamente a 2)
- BOOLEAN.

Triggers

¿Dónde? Tabla qué queremos vigilar.
¿Cuándo? Activar el sensor al meter datos (INSERT), modificar datos (UPDATE), o borrar datos (DELETE).
¿Antes o Después? BEFORE, el trigger se ejecuta antes de que los datos cambien (para validar cosas), o AFTER, el trigger se ejecuta después de que los datos se hayan guardado con éxito (para actualizar otras tablas).

Evento	¿Existe OLD?	¿Existe NEW?	¿A qué representan?
INSERT	No	Sí	**NEW:** Los datos que entran completamente nuevos a la tabla.
UPDATE	Sí	Sí	**OLD:** Los valores antes de modificar. **NEW:** Los valores nuevos que se van a guardar.
DELETE	Sí	No	**OLD:** Los datos de la fila original que se va a eliminar.

Procedures

- Estructura: cabecera, declarar variables, manejo de errores, START TRANSACTION, código a ejecutar, COMMIT

Functions

- Tipos:
  - DETERMINISTIC: la función siempre devuelve el mismo resultado si le pasas los mismos parámetros.
  - NOT DETERMINISTIC: El resultado puede cambiar aunque le pases los mismos parámetros porque depende de valores que mutan en el tiempo (como la hora o números aleatorios).

Tipos de JOIN

- SELECT ... FROM A INNER JOIN B ON A.Clave = B.Clave .Devuelve sólo las filas que tienen coincidencia en AMBAS tablas.
- SELECT ... FROM A CROSS JOIN B. Une cada elemento de A (rombo, triángulo, pentágono) con absolutamente todos los elementos de B (estrella, círculo, cuadrado...). Es una multiplicación (producto cartesiano).
- SELECT ... FROM A RIGHT JOIN B ON A.Clave = B.Clave .Se colorea todo el círculo B (la derecha) y solo la parte de A que coincide. Te asegura que vas a ver **todos los registros de B**, y si alguno no tiene pareja en A, esa parte se rellenará con `NULL`
- SELECT ... FROM A RIGHT JOIN B ON A.Clave = B.Clave WHERE A.Clave is Null .Haces el mismo RIGHT JOIN de antes, pero al añadirle en el `WHERE` que la clave de A sea `NULL`, estás "borrando" la intersección. Sólo elementos de B que no tiene ninguna relación con A
- SELECT ... FROM A FULL OUTER JOIN B ... WHERE A.Clave is Null OR B.Clave is Null .Te quedas con las "orejas" exteriores de ambos círculos. Elimina por completo la parte central.
- SELECT ... FROM A FULL OUTER JOIN B ON A.Clave = B.Clave .El combo completo. Se colorean ambos círculos enteros. Te devuelve absolutamente todo: lo que coincide y lo que no coincide de ambas tablas, rellenando con `NULL` donde falte información.
- SELECT ... FROM A LEFT JOIN B ON A.Clave = B.Clave WHERE B.Clave is Null .Te da **solo lo que está en A pero no está en B**
- SELECT ... FROM A LEFT JOIN B ON A.Clave = B.Clave .Se colorea todo el círculo A (la izquierda) y la intersección.

Popular Base de Datos

Las PK no se tienen en cuenta ya que se ponen automáticamente con el auto increment, pero las FK sí se tienen en cuenta.
Debido a las FK, no puedes insertar los datos en el orden que quieras. Siempre tienes que insertar primero las tablas "padre" (independientes) y luego las tablas "hijo" (dependientes).
Tratar los valores nulos como NULL, y los boolean como TRUE o FALSE.
CURDATE(): Solo la fecha (Sin hora)
NOW(): Fecha + Hora (Estática por consulta)
SYSDATE(): Fecha + Hora (Dinámica en tiempo real)

- INSERTAR. INSERT INTO nombreTabla(...) VALUES (...), (...);

- MODIFICAR. UPDATE nombreTabla SET …;
- ELIMINAR. DELETE FROM nombreTabla WHERE condicion;

CONSTRAINTS en SQL:

1. PRIMARY KEY: Garantiza que el ID sea único y no nulo.
2. FOREIGN KEY: Garantiza que el ID exista en la tabla padre.
3. UNIQUE: Garantiza que no haya valores repetidos (como el email).
4. NOT NULL: Garantiza que el campo no se quede vacío.
5. CHECK: Garantiza que el valor cumpla una condición matemática o lógica.

SELECT DISTINCT codigoPostal FROM Cliente;

Muestra una lista con los códigos postales de los clientes sin repetir.

Diferencia entre where y having
La diferencia es que WHERE filtra uno a uno, elemento a elemento, además de que no se pueden usar funciones de agregación (SUM, COUNT…), escribir WHERE SUM(salario) < 10000, dará error.
HAVING se usa después de agrupar los elementos (después de un GROUP BY), hace el filtro a grupos de datos. Se usa exclusivamente cuando haga falta filtrar usando COUNT, SUM, MAX, AVG…

TIMESTAMPDIFF(YEAR, Clientes.fechaNacimiento, pedidos.fechaRealizacion) >= 18
Hallar los pedidos de los clientes mayores de edad

DATE_SUB(NOW(), INTERVAL '02:15:30' HOUR_SECOND)
Le resta a la hora actual, 2h 15 min y 30 segundos para ser exactos.

Pedidos.fechaRealizacion >= DATE_SUB(CURDATE(), INTERVAL 5 YEAR)

- DATE_SUB(CURDATE(), INTERVAL 5 YEAR): coge la fecha actual y le resta exactamente 5 años
- Luego lo compara con la fecha de realización del pedido, si la fecha del pedido es mayor quiere decir que es más reciente. Recogiendo así los pedidos de los últimos 5 años

WHERE p.fechaRealizacion >= DATE_FORMAT(CURDATE(), '%Y-01-01')

- DATE_FORMAT(CURDATE(), '%Y-01-01'): obtiene el primer día del año actual, cogiendo el año con CURDATE() y escribiendo 01-01 nosotros
- Filtramos así los pedidos que se han realizado únicamente este año

HAVING COUNT(DISTINCT DATE_FORMAT(p.fechaRealizacion, '%Y-%m')) >= 3

- DATE_FORMAT(p.fechaRealizacion, '%Y-%m'): Coge la fecha de cada pedido y "borra" el día, quedándose solo con el año y el mes
- Con DISTINCT eliminamos los repetidos, es decir, de esta forma sólo nos quedan los meses distintos en los que se ha realizado pedido
- **'%Y-%m-%d'** : formato para dejar toda la fecha de forma dinámica.

Para calcular el suelo de una operación: FLOOR(v_facturacionTotal / 5000)

La estructura SELECT ... INTO variable es la forma oficial y nativa de MariaDB/MySQL para guardar el resultado de una consulta dentro de un Procedure:

```
SELECT SUM(lp.precio * lp.unidades)
  INTO v_facturacionTotal
  FROM LineasPedido lp
  JOIN Pedidos p ON lp.pedidoId = p.id
  WHERE MONTH(p.fechaRealizacion) = MONTH(DATE_SUB(CURDATE(), INTERVAL 1
MONTH))
  AND YEAR(p.fechaRealizacion) = YEAR(CURDATE());
```

- MONTH(DATE_SUB(CURDATE(), INTERVAL 1 MONTH)) para calcular el mes anterior

COALESCE(cantidadTotal, 0): La función COALESCE analiza los valores que le pasas de izquierda a derecha y **devuelve el primero que NO sea NULL.** En SQL cualquier operación matemática con NULL, devuelve NULL (10 + NULL = NULL)

- Si cantidadTotal tiene un número (por ejemplo, 10), devuelve 10.
- Si cantidadTotal está vacía (NULL), la función la "transforma" en un 0

p.tipoProductoId <> v_idTipoProductoNuevo
Indica que sean distintos.

DECLARE v_ultimo_cliente ROW TYPE OF TABLE clientes;
Declarar en un PROCEDURE una variable que sea una fila de una Tabla.

EJ: '2026-06-22 11:48:36' . Dinámico: '%Y-%m-%d 00:00:00'
Formato de un DATETIME

```
SELECT
   u.nombre AS nombre_empleado, c.nombre AS nombre_cliente
```

JOIN Usuarios u ON e.usuarioId = u.id . Trae el nombre del **empleado**

JOIN Usuarios c ON cl.usuarioId = c.id .Trae el nombre del **cliente**.

Para traer el nombre del cliente y del usuario en la misma consulta, hacemos dos JOIN diferentes. Además, estamos obligados a usar alias para diferenciar los nombres en el SELECT.

Tipo de objeto	Parámetros	INSERT/UPDATE/DELETE	Llamada	Return
Procedimientos	SÍ	SÍ	CALL sp_name;	NO (1)
Funciones	SÍ	NO (2)	func_name()	SÍ
Triggers	NO (3)	SÍ	NO	NO
Vistas	NO	NO	Select nombre_vista;	NO

(1) Pueden tener parámetros de entrada/salida

(2) Sólo pueden hacer selects

(3) Variables:

- **'old'** (cuando están asociados a eventos de operaciones UPDATE o DELETE). Para acceder a los valores de la fila previos a la actualización o el borrado (valores *antiguos*)
- **'new'** (para eventos de operaciones INSERT o UPDATE). Para acceder a los valores de la fila que se van a insertar o actualizar (valores *nuevos*)